Society for Artificial Intelligence and Deep Learning

SAiDL Induction Assignment 2026

Tracks attempted

Core ML

Long-Context Sequence Modelling

Mechanistic Interpretability

Quantisation and Representation Damage in Sparse Autoencoders

Manit Tanwar

BITS Pilani, Goa Campus

f20230392@goa.bits-pilani.ac.in, tanwarmanit@gmail.com

June 7, 2026

Submitted to the Society for Artificial Intelligence and Deep Learning (SAiDL),
BITS Pilani, Goa Campus.

Code: https://github.com/Manittecool1213/SAiDL-Summer-Assignment-2026

Contents

Introduction

A (not so short) comment on my attempt of the assignment

Disclaimer: This is a rant / heartfelt message. There is absolutely no need to read it; feel free to skip ahead if you want to.

My journey with SAiDL has been a wonky one, especially vis-à-vis the assignment. This is *technically* my third attempt. I have attempted and failed to complete the assignment twice now. I used to start, read up a bunch, and attempt part of a task. Then somewhere along the line I used to feel daunted by the prospect of completing the assignment. Time would pass without progress made. And come the deadline, out of embarrassment, I never even bothered to submit what I had managed to complete.

Over time, it became clearer and clearer to me that a career in ML research is really my calling. So this time around, I had newfound resolve to finally complete the assignment. And it brings me great pride and joy to finally be writing these paragraphs. This time around. . . I did finish it!

The work I was able to do is **far** from perfect. Ultimately, I think no amount of time is enough to work on a project like this — there is just always scope to do more. And to that end, my attempt is fairly minimalistic. Not a lot of bells and whistles, not a lot of optionals attempted; I was more concerned about finishing the whole thing properly. No additional tasks attempted for brownie points.

I have made my fair share of mistakes along the way. Some I was able to rectify before the final deadline, but others remained, and I have done my best to minimise the damage done (more details in the main body of the report itself).

Irrespective of the outcome, I have learned a lot, gained a lot of perspective, and been able to hop on a research trajectory I am happy with. A few things specifically:

- I have never explored Kaggle to the same extent as I did during the course of attempting the assignment this year. I am looking forward to taking part in a few of their contests myself at some point.
- I did not really understand the extent to which tools are freely available to be able to tinker around and do genuinely meaningful and impactful research.
- I am also looking forward to just completing the remainder of the assignment at large. The other questions are also pretty interesting.

Again, irrespective of the outcome of this entire process, a genuinely heartfelt thank you to SAiDL — both the people currently running the show and also all the ones who came before and helped shape the structure of what the induction assignment has come to look like over the years. Ultimately, it was this assignment that really pushed me along in my journey with

ML research from being passive and absorptive to getting my hands dirty (*this* dirty) and trying things out.

1. Core-ML Task

1.1. Baseline

1.1.1. Architecture, Size and Hyperparameters of the Baseline

Architecture

Figures 1 and 2 show the overall architecture and the internals of a single Transformer block respectively. Diagram credit: Claude.

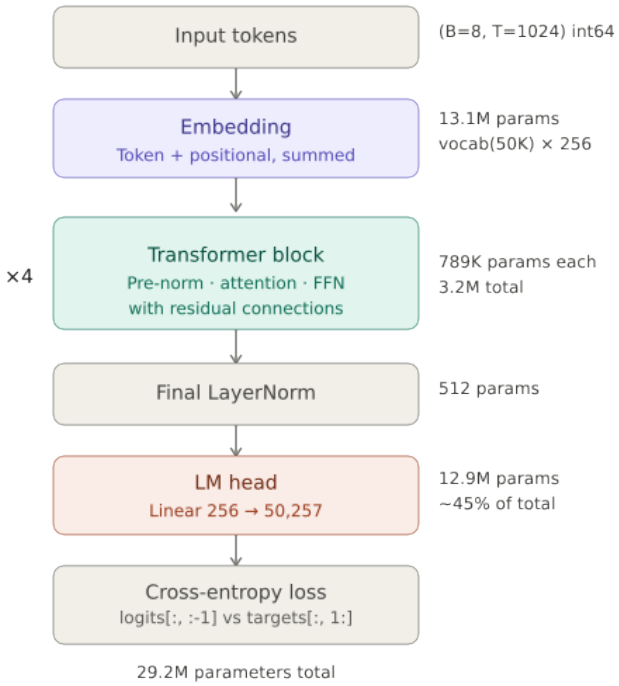

Figure 1. Overview of the baseline Transformer. Input tokens are embedded, passed through four Transformer blocks, and projected to vocabulary logits via the LM head. Total parameter count: 29.2M.

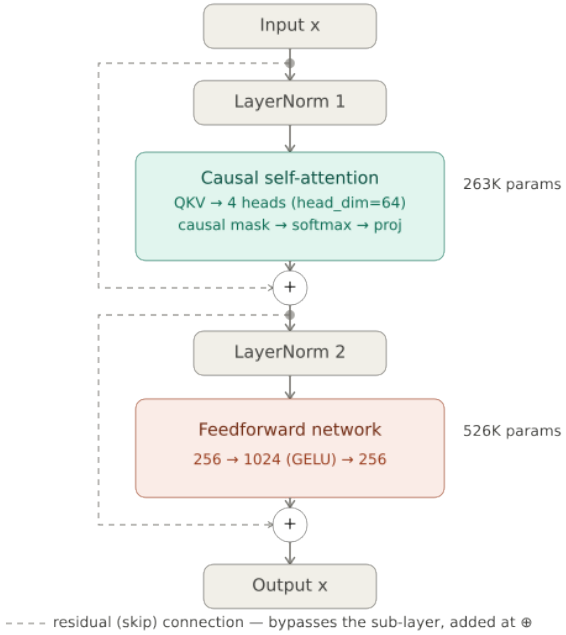

Figure 2. Internals of a single Transformer block. Pre-norm is applied before each sub-layer (causal self-attention; feed-forward network), with residual connections bypassing each sub-layer.

Parameter count

The total parameter count is approximately 29M (exactly 29,203,537), obtained via:

```
1 total_params = sum(p.numel() for p in model.parameters())
```

Table 1 provides a detailed component-level breakdown.

Table 1. Detailed parameter count breakdown for the baseline Transformer.

Component	Formula	Count
Token embedding	$50{,}257 \times 256$	12,865,792
Positional embedding	$1{,}024 \times 256$	262,144
Per block — LN1	2×256	512
Per block — QKV	$256 \times 768 + 768$	197,376
Per block — proj	$256 \times 256 + 256$	65,792
Per block — LN2	2×256	512
Per block — FF1	$256 \times 1{,}024 + 1{,}024$	263,168
Per block — FF2	$1{,}024 \times 256 + 256$	262,400
Per block total		**789,760**
4 blocks	$4 \times 789{,}760$	3,159,040
Final LayerNorm	2×256	512
LM head	$256 \times 50{,}257 + 50{,}257$	12,916,049
Total		**29,203,537**

Chosen hyperparameters and justifications

Core decisions

- `n_embd = 256`:
  - Chosen to limit memory.
  - The FFN expands to $4 \times$ `n_embd` internally, and attention scales as $O(\texttt{n\_embd}^2)$.
  - Doubling to 512 would quadruple costs.
  - As a comparison, GPT-2 uses 768. This is a deliberate scaling down for training from scratch on limited hardware.
- `n_head = 4`:
  - This results in a head dimension of $256/4 = 64$ — the standard used across GPT-2 and GPT-3.
  - No major reason for this particular choice. It just divides `n_embd` evenly, and results in the same head dimension as in established works.
- `n_layer = 4`:
  - Again, no hard reasoning for this choice. It could just as well have been slightly larger or smaller.
  - A smaller value was avoided because deeper models intuitively outperform shallower ones on language modelling.
  - A larger value was avoided to keep epoch duration at bay.
- `block_size = 1024`: Mandated by the assignment.
- `batch_size = 8`:
  - This is the largest batch size that fits given the remaining constraints at a context length of 1024.
  - Anything larger might have exceeded the Colab free tier 16 GB VRAM capacity.

Standard picks

- `learning_rate` $= 3 \times 10^{-4}$: AdamW standard for this scale, as used by Karpathy in nanoGPT (Karpathy, 2022).
- `dropout = 0.1`: As used in Vaswani et al. (2017) and subsequent works.
- `seed = 42`: Just for reproducibility. More on this in Section 1.1.4.

1.1.2. Deviation from the *Attention Is All You Need* Architecture

The assignment asks to implement a standard Transformer model *as in* the famous Attention paper. The implemented architecture **does** retain all of the essential components, but makes two notable deviations.

Learned positional embeddings instead of sinusoidal positional encodings

- The former became a common alternative shortly after the original paper.

- They offer a simpler implementation, and have been used by several autoregressive language models (e.g. GPT-2).

Pre-LayerNorm instead of Post-LayerNorm

- This too has become standard practice after the original paper.
- Pre-LN offers better gradient flow through residual paths and a reduced risk of training instability (Xiong et al., 2020).

1.1.3. Epoch Count Probe and Early Stopping

I initially was not sure how long I should train the baseline for (and consequently all the remaining variants too). At this point, I was not super familiar with Kaggle, and was defaulting to Colab, which is notoriously problematic for long runs.

Consequently, I started by performing an epoch count probe on the baseline, with early stopping based on a patience mechanism on validation perplexity. This strategy makes the following assumptions:

- Learning takes place in two stages. In the first stage, the model is learning the underlying signal in the data. In the second stage, after all the major patterns have been learned, the optimiser continues to reduce training loss, but through undesirable overfitting.
- The model is unlikely to learn new patterns later in the training phase.

It is not possible to theoretically guarantee that the model will not improve later on, but for training with limited hardware, I think such a probe is justifiable. Ultimately, I treated validation perplexity as a generalised measure of model performance, and stopped training when validation perplexity had been increasing for three consecutive epochs.

The initial probe was performed without a seed, as its only purpose was to gauge epoch count. Figure 3 shows the resulting curve; the shaded region marks where perplexity increased for three consecutive epochs.

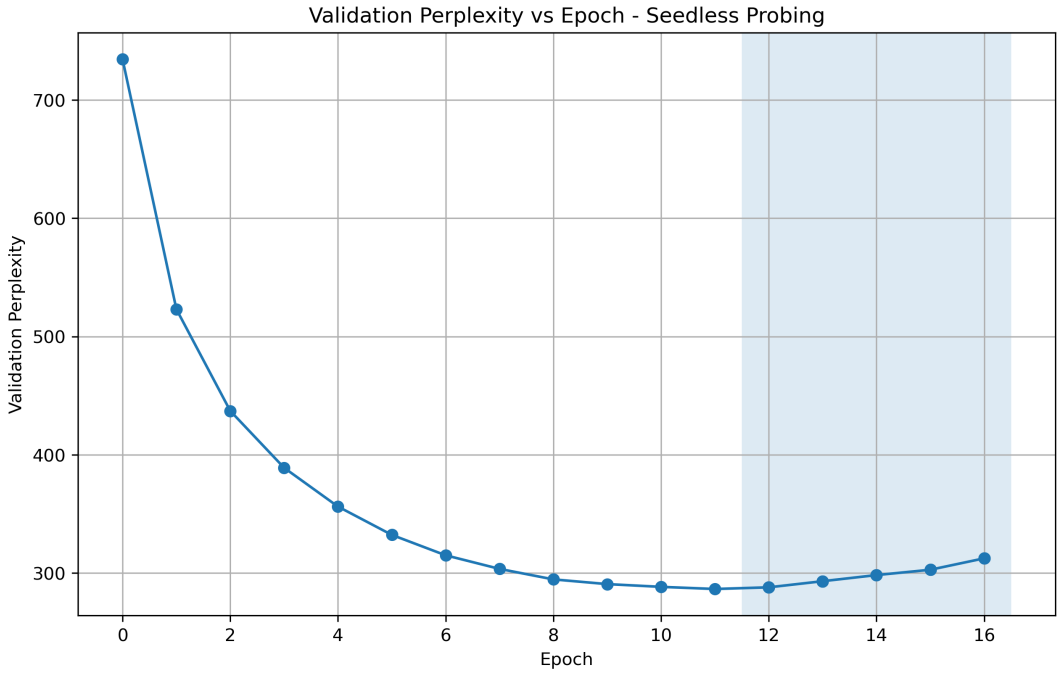

Figure 3. Validation perplexity vs. epoch during the seedless epoch count probe. The highlighted region to the right marks three consecutive epochs of increasing validation perplexity, triggering early stopping. The best epoch in this run was epoch 11.

From this run, the best epoch was epoch 11. I maintained an epoch threshold of 15 for all remaining experiments. I considered updating this, but the same pattern was followed by subsequent experiments, making said change unwarranted.

1.1.4. Disclosure on Single Seed

All experiments for the Core-ML task were performed with a common seed (42). Ideally, these should have been repeated with multiple seeds to prevent artefacts of getting a lucky seed. This was not done because of time constraints.

1.1.5. Results

Perplexity

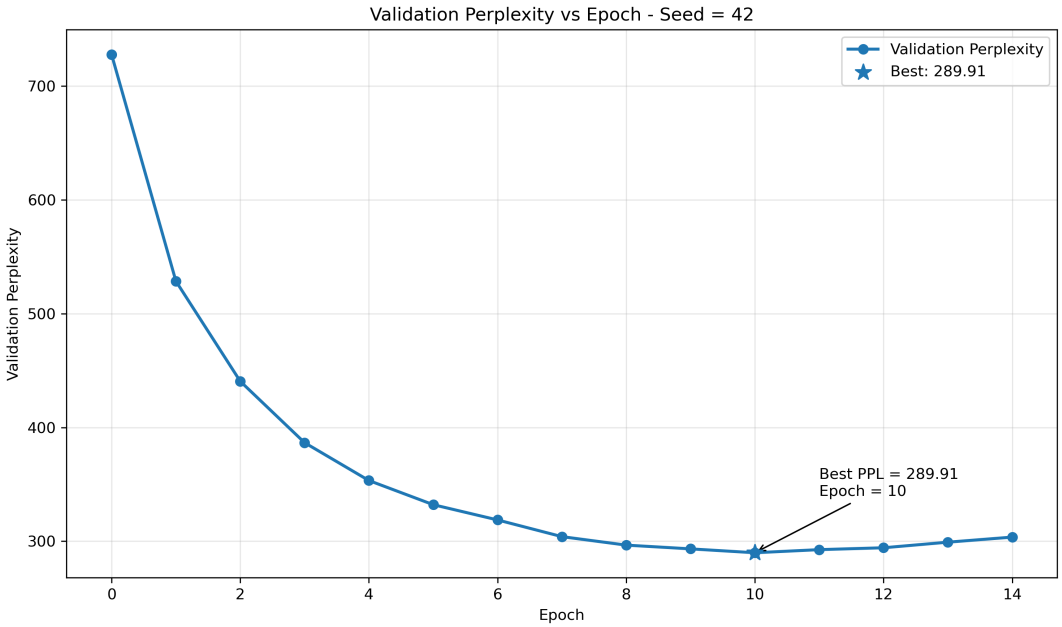

Figure 4. Validation perplexity vs. epoch for the baseline Transformer (seed = 42, ctx = 1024). Best validation perplexity: **289.91** at epoch 10.

Throughput

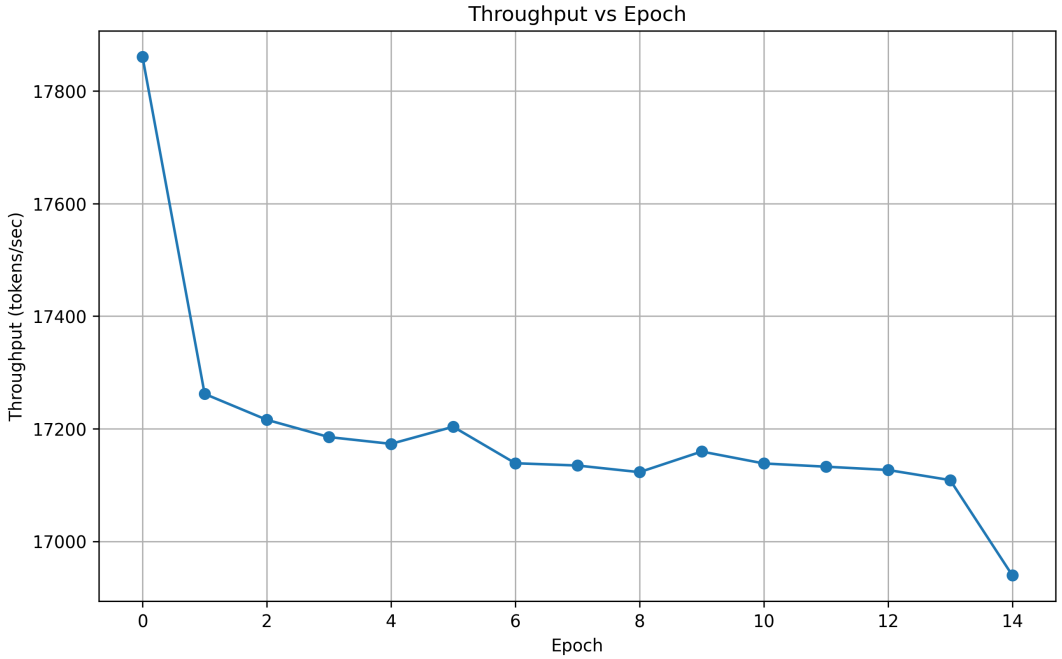

Figure 5. Training throughput (tokens/sec) vs. epoch for the baseline Transformer. Throughput is stable to within 2% after the 0^{th} epoch.

Observations and reasoning

- Average throughput:

  - Including 0^{th} epoch: 17,193.51 tokens/sec
  - Excluding 0^{th} epoch: 17,145.84 tokens/sec
- Maximum throughput: 17,860.9 tokens/sec
- Minimum throughput: 16,939.6 tokens/sec
- Throughput remains more or less constant after the 0^{th} epoch, with less than 2% variation.
- The first epoch:
  - This is actually atypical; owing to hardware warmups, one would expect the first epoch to have *lower* throughput, not higher.
  - This is likely a measurement artefact or a transient hardware effect rather than a training effect; to that end, I am choosing not to over-interpret this.

1.2. Attention Variants

Variants chosen:

- Sliding window
- Multi-Query (MQA)
- Linear Attention (kernelised)

1.2.1. Validation Perplexity and Training Stability

In all plots, the light blue shaded region towards the right marks where validation perplexity increases instead of decreasing. The best-performing variant at each epoch is marked with a star (⋆).

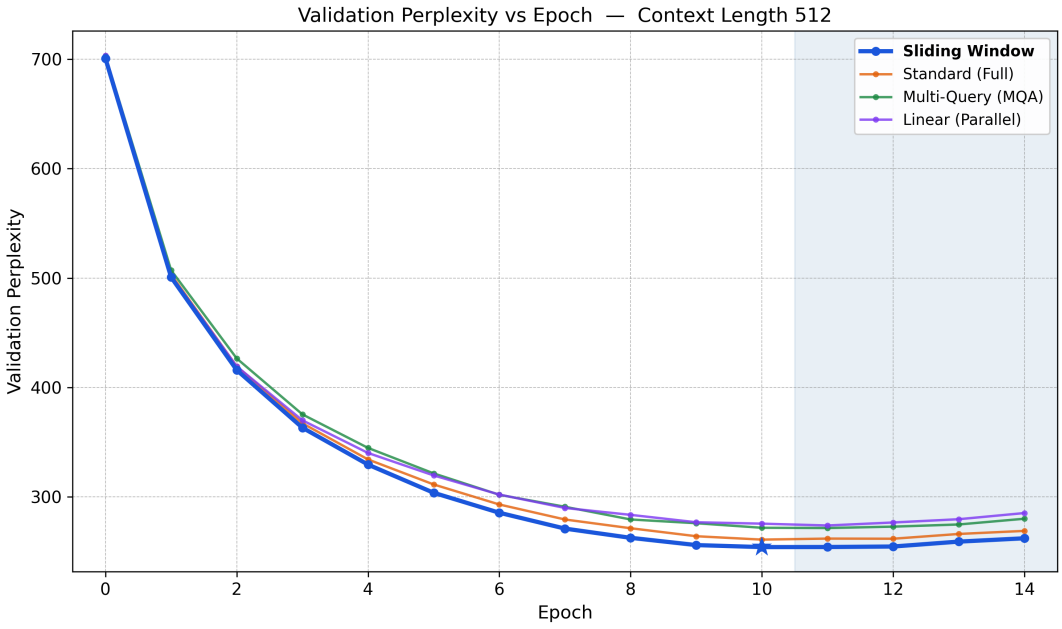

Figure 6. Validation perplexity vs. epoch for all attention variants at context length 512.

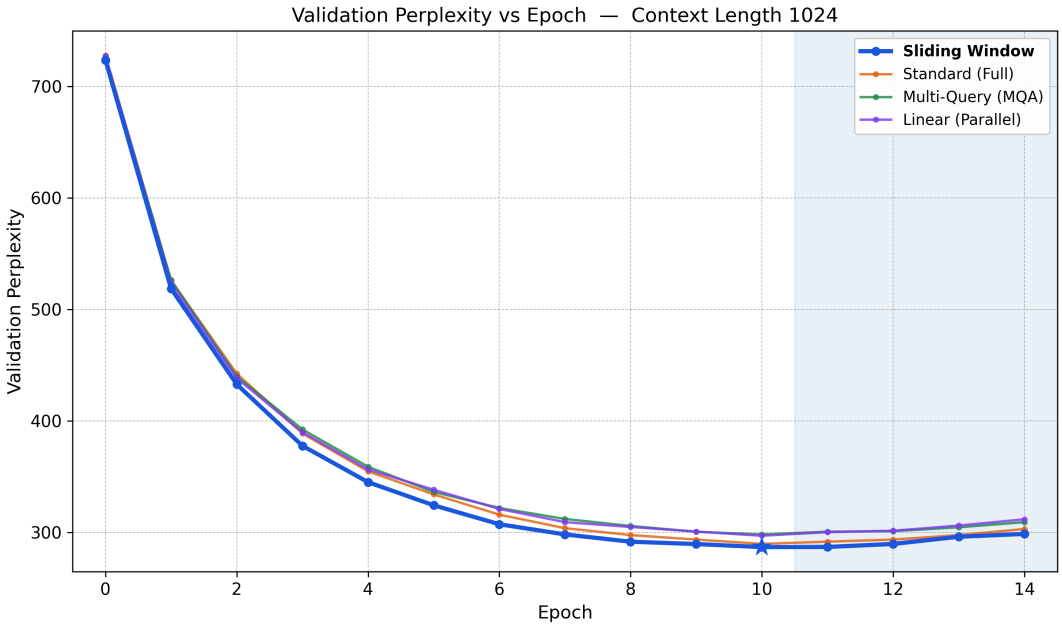

Figure 7. Validation perplexity vs. epoch for all attention variants at context length 1024.

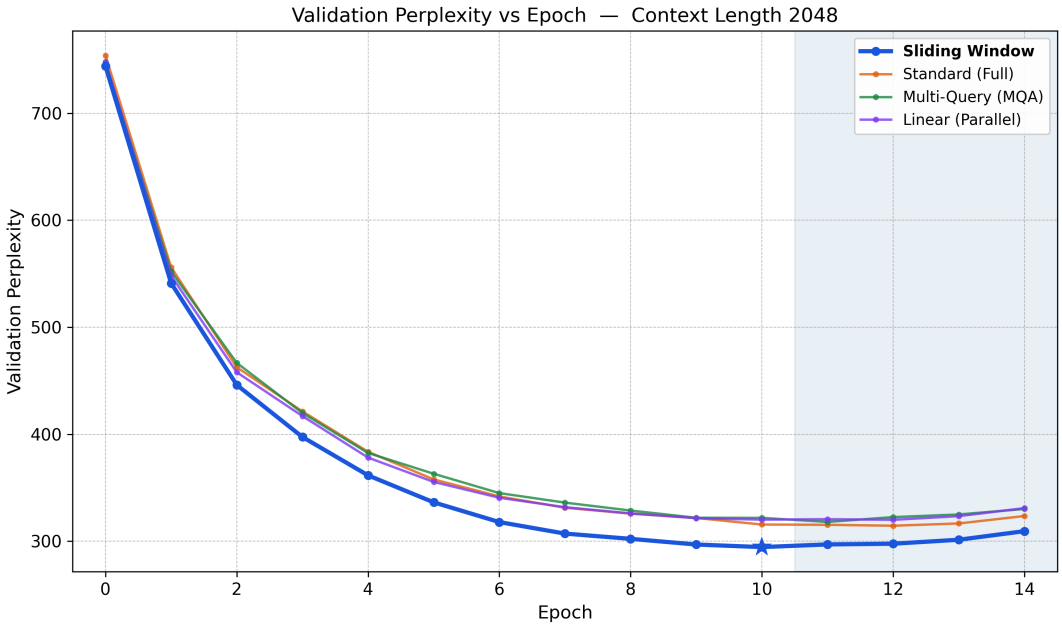

Figure 8. Validation perplexity vs. epoch for all attention variants at context length 2048.

Observations and reasoning

- At **every** context length, sliding window attention outperforms other variants.
- At longer context lengths, sliding window's delta over other variants is larger. Gaps in best-epoch perplexity between sliding window and the next-best variant:
  - ctx = 512: 6.8
  - ctx = 1024: 2.9
  - ctx = 2048: 20.0

This is somewhat surprising. Sliding window should ideally underperform full attention on tasks requiring long-range dependencies, and WikiText-2 *does* contain passages with such dependencies. Two possible explanations:

  - The model might itself be too small to exploit long-range dependencies meaningfully.
  - **Implicit regularisation:** By restricting each token to attend only to a local window, sliding window prevents the model from fitting spurious long-range correlations in the training data. The local constraint serves as an inductive bias, avoiding noisy patterns and improving generalisation. This effect would compound at longer context lengths.

- All four variants converge at roughly the same rate in the first five epochs, then separate:
  - The divergence happens around the slow convergence phase (epochs 5–10), rather than during the initial rapid drop.
  - This suggests that the difference between variants is not one of *learning speed*, but rather of *quality of representation learned*.
  - If sliding window were just faster than the remaining variants, they would all have converged to similar perplexity values at later epochs. Instead, they each converge to a separate plateau.
- Multi-query attention sits between standard full attention and linear attention at most context lengths, which is the expected result — it reduces the KV heads to one, saving memory and compute, but at some cost to the richness of the attention patterns each head can represent.

1.2.2. Peak GPU Memory Consumption

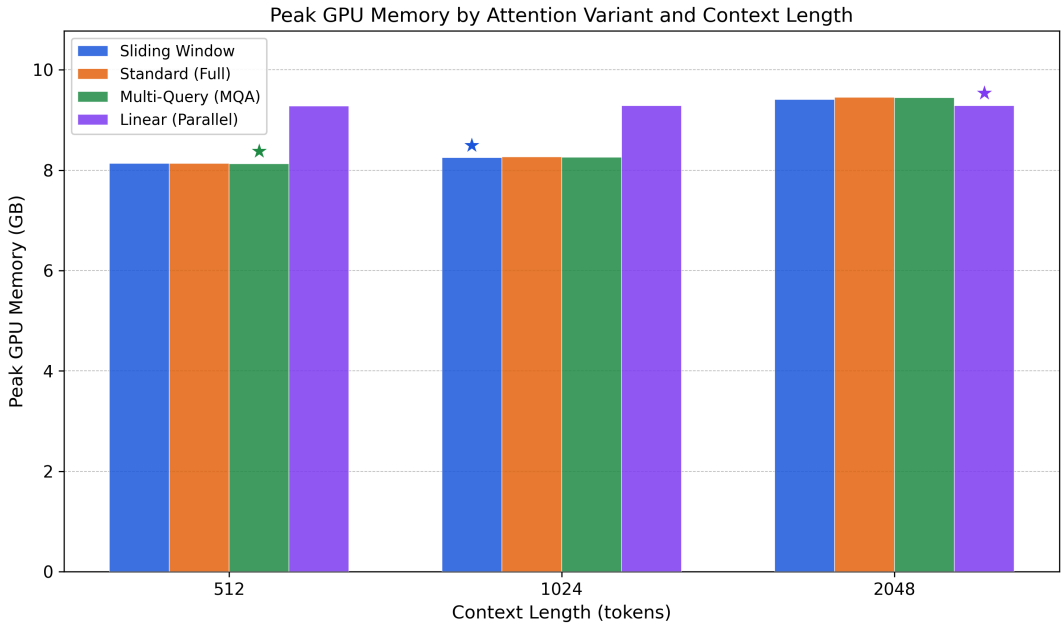

Figure 9. Peak GPU memory (GB) vs. context length for all four attention variants. Linear attention's flat profile contrasts with the growing memory footprint of the softmax-based variants.

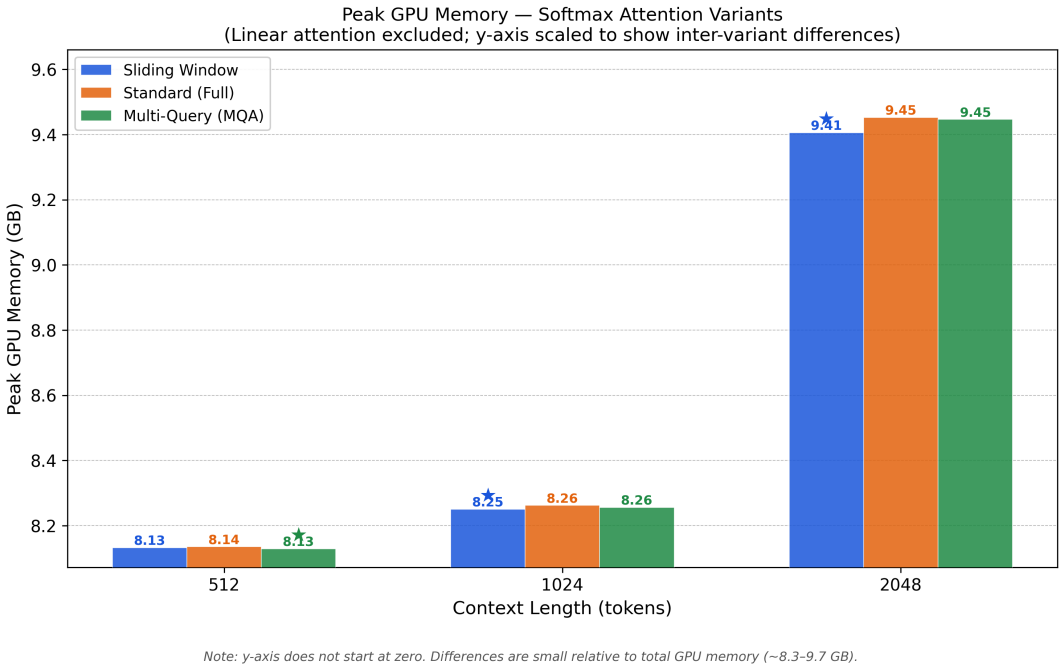

Figure 10. Peak GPU memory for the three softmax-based variants (linear attention excluded), with a scaled Y-axis to highlight inter-variant differences. Note: the Y-axis does not start at zero. Differences are small relative to total GPU memory (≈8.3–9.7 GB).

Observations and reasoning

- Linear attention's memory profile is more or less constant across all context lengths, clearly demonstrating $O(1)$ (in context length) memory scaling.
- The crossover point is already visible: linear attention consumes *more* memory than the softmax-based variants at ctx = 512 and 1024, but becomes competitive at ctx = 2048. Any further increases in context length would increase the margin that linear attention has over the other variants.
- Sliding window, standard full, and multi-query attention *do* have minor variations in their memory usage owing to their different architectures. However, the attention-specific memory is relatively small compared to the remainder of the model, which remains unchanged across variants, thereby leading to an imperceptible difference on the total memory scale.
- Linear attention has an additional `kv_cumsum` tensor which consumes significant memory relative to the rest of the architecture, and must persist throughout the forward and backward passes, thereby inflating the required memory.

1.2.3. Inference Latency

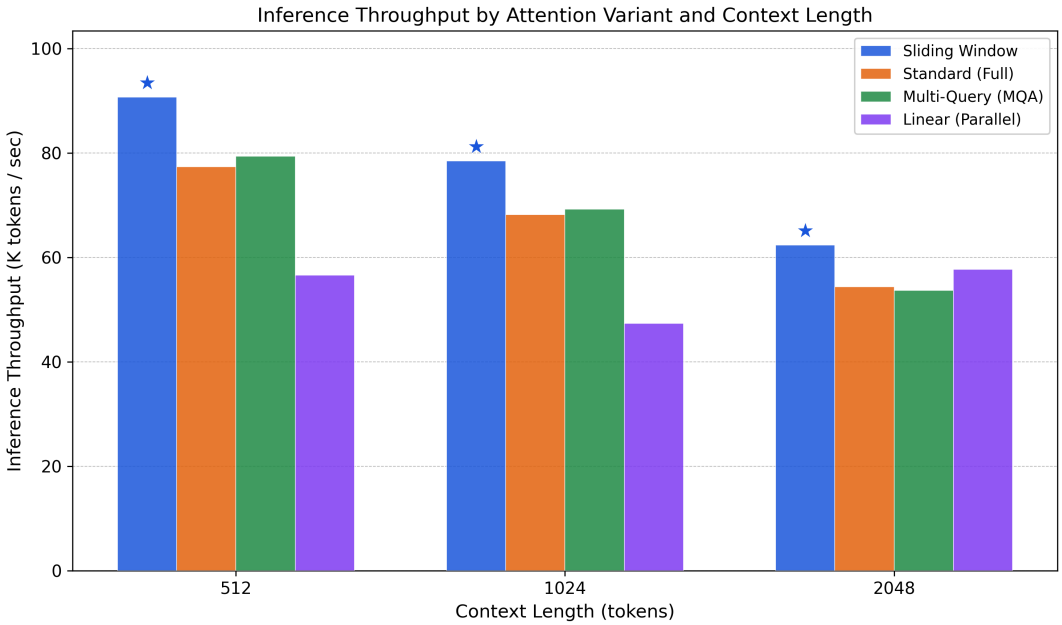

Figure 11. Inference throughput (K tokens/sec) vs. context length for all four attention variants. Sliding window is the consistent frontrunner.

Observations and reasoning

- Sliding window is the consistent frontrunner, but the gap between it and other variants *decreases* with increasing context length.
  - This is counterintuitive; its $O(T \cdot w)$ complexity should outperform $O(T^2)$ methods at longer context lengths.
  - One possible reason is that at larger context lengths, all variants become more memory-bandwidth constrained. The GPU may be spending more time on data movement than on computation, which limits the benefit of a cheaper computational complexity.
- Linear attention's throughput is lowest despite its $O(T)$ complexity:
  - This is likely another consequence of the `kv_cumsum` operation, which creates a large intermediate tensor that stresses memory bandwidth.
  - At the scale of these sequence lengths, the constant factor is not small enough to counteract the asymptotic savings.
  - This suggests that the crossover point for linear attention vs. other variants in terms of throughput lies at even larger context lengths.

1.3. Positional Embedding Variants

Extrapolation performance (all models trained at $L_{\text{train}} = 512$, evaluated at $L_{\text{eval}} \in \{512, 1024, 2048\}$):

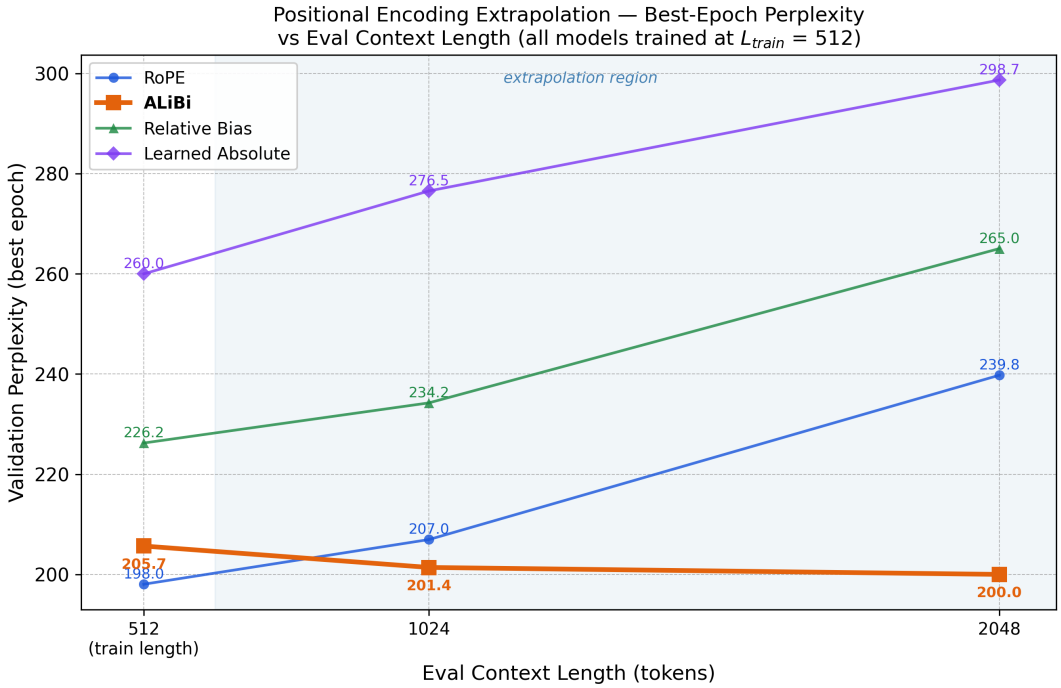

Figure 12. Best-epoch validation perplexity vs. evaluation context length for all four positional encoding variants. All models are trained at context length 512; the shaded region marks the extrapolation regime. ALiBi is the only variant whose perplexity does not increase — and in fact decreases — with longer evaluation contexts.

In-distribution training performance (eval ctx = train ctx = 512):

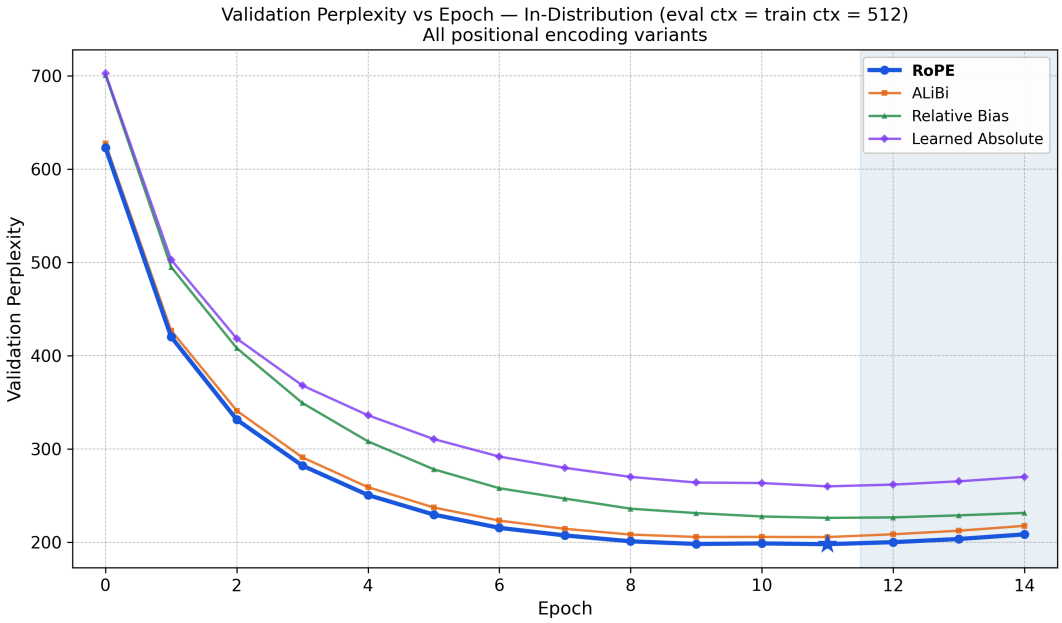

Figure 13. Validation perplexity vs. epoch for all positional encoding variants, with evaluation context length equal to training context length (512). All variants converge at nearly identical rates.

Observations and reasoning

- ALiBi is the only variant that does not degrade — and actually improves — with increasing context length.
  - This improvement is a direct consequence of what ALiBi's positional representation actually is.
  - Every other method stores positional information in a table of some kind — a lookup for a position index, a rotation angle for a frequency band, or a bucket for a distance.
  - All of these tables have a training domain, and positions outside that domain produce either zero, a poorly trained entry, or an extrapolated angle that the model's weights were not calibrated against.
  - ALiBi has no table. Its bias at any position pair (i, j) is simply $\big(-m_h \cdot (i-j)\big)$, where m_h is a fixed per-head slope that is not learned and requires no training. This expression is defined for all non-negative integers and is not restricted to a particular context length.
- ALiBi, instead of degrading, slightly improves as context length increases.
  - This is likely a consequence of the chosen context lengths. I would not expect this to generalise to even longer context lengths.
  - Because the training context length was just 512, the model likely benefited meaningfully from the additional context which ALiBi was able to effectively provide at the longer testing context lengths.
- None of the three non-ALiBi variants extrapolate well.
  - While absolute perplexity values follow the ordering RoPE < Relative Bias < Learned Absolute, perplexity alone is not the best metric upon which to judge extrapolation capability.
  - The percentage increases in perplexity are more or less the same across all three variants — no one extrapolates meaningfully better than the other.
- From the in-distribution curve: all four variants converge at nearly identical rates and reach their best epoch at the same point.
  - This means that the differences in extrapolation ability cannot be attributed to 'better' in-distribution learning.
  - They all learn equally well in-distribution; the divergence is solely a property of how each variant handles positions it was never trained on.

1.4. Convolution + Attention Hybrids

Best-performing components selected from previous sections:

- **Attention:** Sliding Window
- **Positional Embedding:** ALiBi

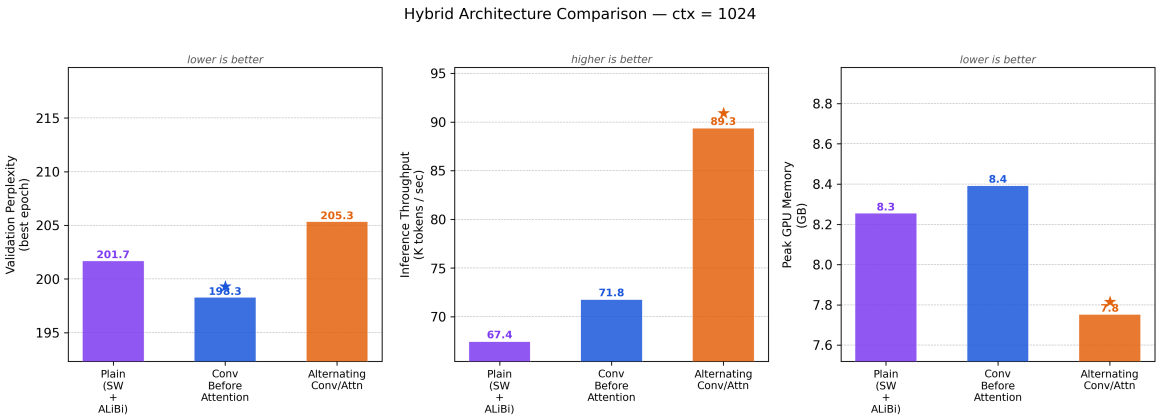

Figure 14. Side-by-side comparison of all three hybrid configurations at ctx = 1024 across validation perplexity (lower is better), inference throughput (higher is better), and peak GPU memory (lower is better). Note: none of the Y-axes start from zero; refer to data labels for absolute values.

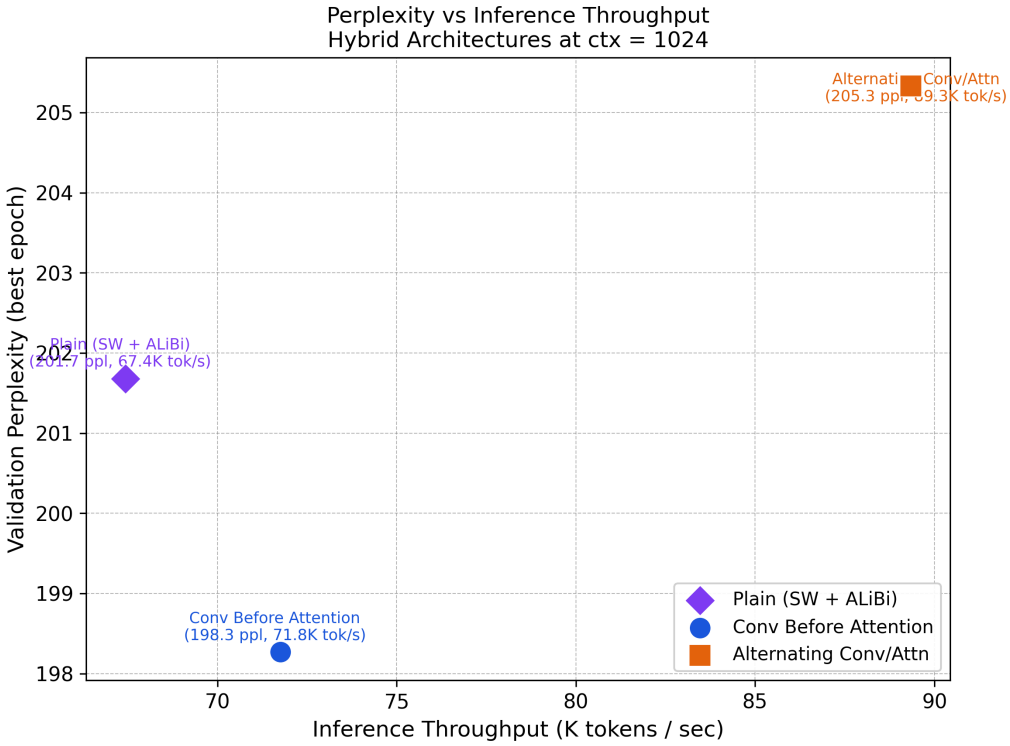

Figure 15. Perplexity vs. inference throughput for all three hybrid configurations at ctx = 1024. The bottom-right region is most desirable (lower perplexity, higher throughput). Due to the scaled axes, differences appear larger than they are; the bar charts in Figure 14 are more representative of absolute magnitudes.

Observations and reasoning

- First and foremost, it is worth noting that the plots exaggerate deltas visually. The actual gaps across all metrics are small, with the only truly large gap being throughput, where the leader has a 17-point lead over the next-best variant. To that end, one could argue that the interleaved Conv + Attention hybrid is the best of the three — it sacrifices some perplexity for a massive increase in throughput.

- **Convolution before attention achieves the best perplexity.**
  - In this hybrid, the convolutional branch runs as a separate residual contribution before every attention block. Consequently, the token representations entering attention have already been 'enriched' by local convolutional filtering.
  - Every layer benefits from this, resulting in an overall advantage in perplexity.
- **Alternating conv/attention achieves the highest throughput by a margin.**
  - In this hybrid, attention layers at positions 1 and 3 are replaced entirely by convolution.
  - Depthwise-separable convolution is **substantially** cheaper than self-attention, and consequently runs much faster.
  - This speedup comes at an intuitive perplexity cost: replacing attention entirely loses the global context that it provides.
- Conv before attention performs somewhat better than the plain variant on throughput:
  - This is counterintuitive at first — if we are adding a convolutional branch to every block, should compute not increase?
  - A likely explanation is that the convolutional operation, despite adding computation, is extremely cache- and parallelism-friendly. In comparison, sliding window attention, despite being faster than regular attention, still involves a masked softmax and irregular memory access patterns which are difficult to parallelise.
  - **An important caveat:** the observed differences in throughput are ultimately small and could be attributed to inter-run variance. Because of the small scale of the differences and the lack of additional experimental data, this inference is to be treated with some scepticism.
- All three variants somewhat lie on a Pareto frontier between perplexity and computation.
  - This is not very evident from the plot owing to the sparsity of data, but seems like an intuitive claim.
  - One can choose to design for speed at the cost of perplexity, or vice versa; there is no universal outperformer.
  - This is consistent with what we see in practice: some ('flash') models are faster but produce lower quality responses, whereas other ('thinking') models are slower but produce more meaningful ones.

1.5. Comparative Tables

The following three tables report unified metrics across all experiments. Best-epoch validation perplexity is reported throughout; efficiency metrics are measured at the corresponding best epoch.

Table 2. Attention variant results across context lengths. **Val PPL** is the best-epoch validation perplexity (minimum across all epochs); all other metrics are reported at the final epoch. ⋆ denotes the best value in each column within each context-length group.

Ctx	Variant	Train Loss	Val Loss	Val PPL ↓	Peak Mem (GB) ↓	Epoch Time (s) ↓	Inf Tput (K t/s) ↑
512	Sliding Window⋆	4.430	5.538	**254.1**	**8.13**	**110.4**	**90.7**
	Standard (Full)	4.456	5.564	260.9	8.14	119.5	77.4
	Multi-Query (MQA)	4.400	5.604	271.5	**8.13**	117.9	79.4
	Linear (Parallel)	4.382	5.613	273.8	9.28	165.9	56.6
1024	Sliding Window⋆	4.561	5.659	**286.7**	**8.25**	**122.1**	**78.5**
	Standard (Full)	4.600	5.669	289.6	8.26	131.4	68.2
	Multi-Query (MQA)	4.637	5.697	298.0	8.26	129.9	69.2
	Linear (Parallel)	4.614	5.693	296.9	9.28	189.0	47.4
2048	Sliding Window⋆	4.647	5.685	**294.5**	9.41	**144.0**	**62.4**
	Standard (Full)	4.532	5.751	314.5	9.45	155.5	54.3
	Multi-Query (MQA)	4.671	5.762	318.1	9.45	158.9	53.7
	Linear (Parallel)	4.541	5.768	320.0	**9.29**	161.4	57.7

Table 3. Positional encoding results. All models trained at $L_{\text{train}} = 512$. **Val PPL** is the best-epoch validation perplexity at each eval context length. Memory, epoch time, and throughput are measured at the training context length and are identical across eval context lengths for each variant. ⋆ denotes the best value in each column within each eval-context group.

Eval Ctx	Variant	Train Loss	Val Loss	Val PPL ↓	Peak Mem (GB) ↓	Epoch Time (s) ↓	Inf Tput (K t/s) ↑
512	RoPE⋆	4.035	5.288	**198.0**	7.81	123.9	76.1
	ALiBi	4.058	5.326	205.7	7.93	124.0	74.9
	Relative Bias	4.217	5.422	226.2	**7.69**	126.0	72.3
	Learned Absolute	4.345	5.561	260.0	**7.69**	**119.9**	**80.0**
1024	ALiBi⋆	4.058	5.305	**201.4**	7.93	124.0	74.9
	RoPE	4.271	5.333	207.0	7.81	123.9	76.1
	Relative Bias	4.217	5.456	234.2	**7.69**	126.0	72.3
	Learned Absolute	4.577	5.622	276.5	**7.69**	**119.9**	**80.0**
2048	ALiBi⋆	4.058	5.298	**200.0**	7.93	124.0	74.9
	RoPE	4.271	5.480	239.8	7.81	123.9	76.1
	Relative Bias	4.460	5.580	265.0	**7.69**	126.0	72.3
	Learned Absolute	4.577	5.699	298.7	**7.69**	**119.9**	**80.0**

Table 4. Convolution–attention hybrid results at context length 1024. **Val PPL** is the best-epoch validation perplexity; all other metrics are reported at the final epoch. ⋆ denotes the best value in each column.

Design	Train Loss	Val Loss	Val PPL ↓	Peak Mem (GB) ↓	Epoch Time (s) ↓	Inf Tput (K t/s) ↑
Plain (SW + ALiBi)	4.223	5.307	201.7	8.25	130.7	67.4
Conv Before Attention	4.156	5.290	**198.3**	8.39	127.6	71.8
Alternating Conv/Attn	4.259	5.325	205.3	**7.75**	**109.9**	**89.3**

2. Mechanistic Interpretability Task

Why I chose this task

Frankly, this was not part of the original plan. I originally intended to do the diffusion task, and did work on that for a while. However, the task was taking too long for me to complete, so I considered a pivot. Mechanistic interpretability then caught my eye, in some part due to the Welch Labs Grokking video, which I watched around the time I considered the pivot.

2.1. Pipeline Setup

2.1.1. Disclosure on Data Scale

Instead of performing streaming with 80–100M tokens, I have only used 10M. Some arithmetic:

- Activations are stored as float16; each token produces a 768-dimensional vector.
- The required storage is thus `N_tokens` $\times$ 768 $\times$ 2 bytes = `N_tokens` $\times$ 1,536 bytes.
- For 80M tokens, this would require $\approx$122.9 GB.
- For 10M tokens, this would require $\approx$15.4 GB.

Kaggle's `/kaggle/working` directory gives 20 GB, meaning 80M tokens would not fit on the available disk space. 10M tokens fit with a healthy margin.

I am aware that the assignment asks to use streaming, which would bypass all disk constraints in the first place. However, I was hesitant to do this for the following reasons:

- This approach would require extraction and training to happen in the same session — one which would likely take a sizeable chunk of time.
- When I started this task, I had only just shifted to Kaggle from Colab, and had not had much experience with Kaggle in the past. At the time, I did not know that Kaggle is significantly more stable and reliable than Colab in terms of running experiments.
- I was afraid of these runs taking an extremely long duration and failing midway, as had been the case for me on multiple separate occasions when working on the Core-ML task with Colab.
- Owing to my shift from the Diffusion task to this one, I had relatively limited time. While this was still sufficient to complete the task, I did not want to risk having to repeat runs. I consequently went with the safer but suboptimal choice of restricting the dataset.

While there certainly could have been artefacts which only became evident when using more data, there are some positive indicators suggesting that choosing a smaller subset of the data did not cause much harm.

The training loss curve

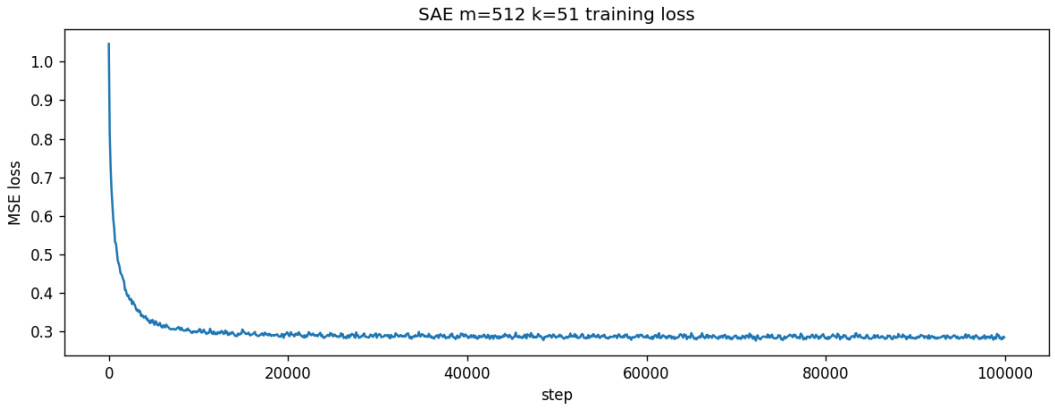

Figure 16. SAE ($m = 512$, $k = 51$) reconstruction loss over 100k training steps. The loss converges well before the 100k step mark, suggesting the 10M-token training set is not a binding constraint on convergence.

- The SAE loss had converged well before the 100k step mark.
- Had the SAE not converged, it would have been a strong indicator that the data were insufficient.
- The loss meaningfully decreased using the dataset size I opted for.

Relative instead of absolute comparisons for downstream analysis

- For the later analysis sections, absolute SAE quality matters less than relative comparisons.
- Questions such as 'Does 8-bit quantisation damage representations less than 2-bit?' and 'Does the sparsity fragility result hold?' are robust to SAE quality because they are made within the same experimental setup.

2.1.2. Varying Bottleneck Sizes

Two SAE bottleneck widths were trained:

- $m = 512$ ($k = 51$ active features per token), and
- $m = 1024$ ($k = 102$),

both for 100k steps with identical hyperparameters. All analyses in subsequent sections are conducted on both variants, with comparative results reported where they differ meaningfully.

2.1.3. SAE Training Health

Training health was monitored via three metrics logged every 100 steps:

- Reconstruction loss.
- L0: the number of non-zero entries in the SAE's bottleneck activation vector for a given input token.
- Percentage of dead features.

L0 remained exactly equal to k throughout training, confirming that the top-k constraint was enforced at every step. Dead features also remained near zero throughout the entire training run, indicating that the SAE's dictionary was being fully utilised.

2.2. Quantisation Analysis

2.2.1. Methodology

Some portion of this subsection restates or elaborates on terms present in the assignment. I have included this because this was one of the newest areas for me; there were a **lot** of terms I was unfamiliar with.

Min-max calibration

- This is the method through which the quantisation step size is determined — i.e., the gap between adjacent quantisation levels.
- For per-tensor quantisation, one global step size is produced.
- For per-feature quantisation, 512 separate values are produced, one per feature dimension.

Metrics used

- **Perplexity:**
  - Measures downstream task impact of replacing layer 3 activations with their quantised counterparts.
  - This is an important metric — it directly measures whether the model can still do its job.
- **MSE:**
  - Measures raw numerical distortion.
  - Asks: how different are the quantised activation values from the original values on average?
- **CKA (Centred Kernel Alignment):**
  - Measures geometric similarity.
  - Asks whether the overall shape and structure of the representation space is preserved.
  - It is a relative measure — two representations can have high MSE but also high CKA if they are related by a scaling transformation.
- **SDS (Subspace Distortion Score):**
  - Measures how much of the quantisation error falls specifically within the most information-carrying directions of the activation space — the top singular vectors.
  - Low SDS means the error is concentrated in directions the model cares less about. High SDS means the error is destroying the most important structure.

2.2.2. Results

Table 5. Quantisation analysis results for $m = 512$ and $m = 1024$. All ΔPPL% values are relative to the clean distilgpt2 baseline (≈36.9). The identity patch row represents the SAE's own reconstruction overhead with no quantisation applied. VQ bit rates are per activation *vector* rather than per dimension and are not directly comparable to uniform bitwidths.

		$m = 512$				$m = 1024$			
Quant type	**Bits**	**PPL**	**ΔPPL%**	**CKA**	**SDS@64**	**PPL**	**ΔPPL%**	**CKA**	**SDS@64**
Clean baseline	—	≈36.9	—	—	—	≈36.9	—	—	—
Identity patch	—	59.79	+62.2	1.000	0.000	47.03	+27.6	1.000	0.000
Per-tensor	8-bit	59.65	+61.9	0.905	0.030	46.82	+27.1	0.995	0.004
Per-tensor	4-bit	92.33	+150.5	0.848	0.101	52.19	+41.6	0.978	0.025
Per-tensor	2-bit	2289.3	+6111.5	0.519	0.702	336.02	+811.7	0.740	0.393
Per-feature	8-bit	61.76	+67.6	0.936	0.060	48.76	+32.3	0.978	0.033
Per-feature	4-bit	63.63	+72.7	0.919	0.077	49.81	+35.1	0.972	0.044
Per-feature	2-bit	109.07	+196.0	0.797	0.212	80.12	+117.4	0.902	0.151
VQ (256 codes)	≈8b/vec	555.88	+1408.3	0.924	0.235	572.56	+1453.5	0.807	0.282
VQ (1024 codes)	≈10b/vec	400.04	+985.4	0.940	0.186	412.51	+1019.3	0.843	0.230

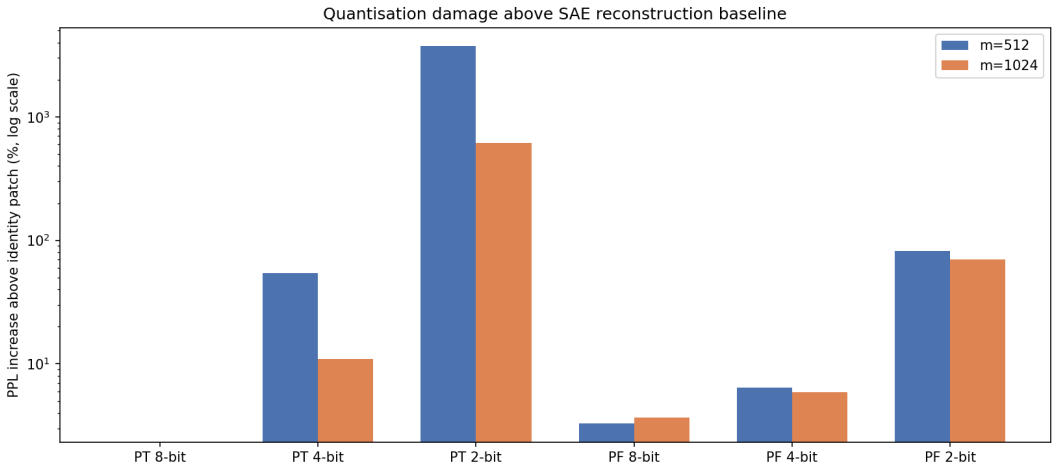

Figure 17. PPL increase above the SAE reconstruction baseline (identity patch) for each quantisation scheme, shown on a log scale. Blue: $m = 512$; orange: $m = 1024$. Per-tensor 2-bit and both VQ variants are catastrophically worse than all others.

2.2.3. UMAPs

$m = 512$**:**

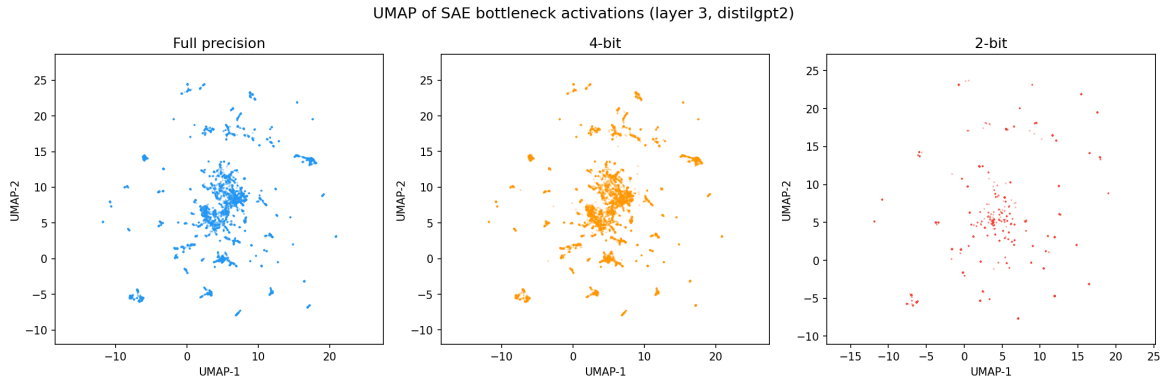

Figure 18. UMAP projections of SAE bottleneck activations for $m = 512$ under full-precision (blue), 4-bit per-tensor (orange), and 2-bit per-tensor (red) quantisation. The UMAP is fit on the full-precision activations; quantised representations are projected into the same embedding. Geometric collapse is visible at 2-bit.

$m = 1024$**:**

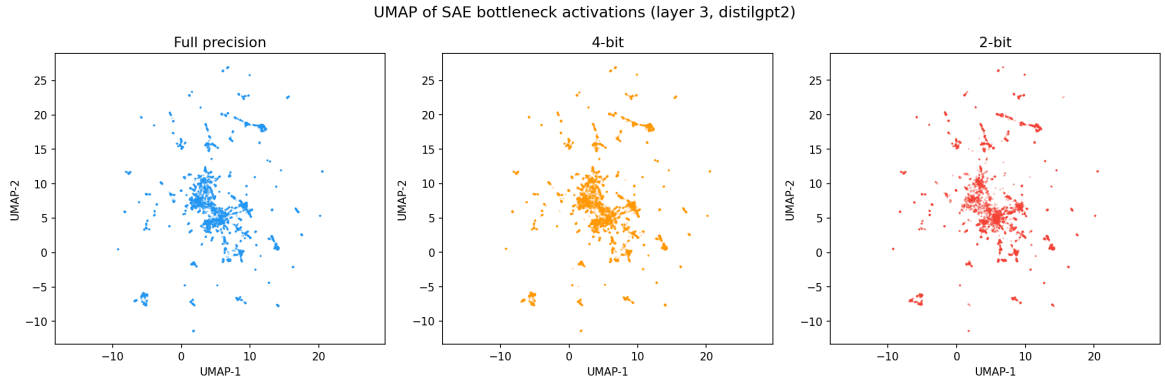

Figure 19. UMAP projections of SAE bottleneck activations for $m = 1024$ under the same three quantisation settings as Figure 18. The larger dictionary shows visibly less geometric distortion at 4-bit compared to $m = 512$.

2.2.4. Observations and reasoning

The identity patch

- It has a PPL of ≈59.79, which is +62.2% over the clean baseline of ≈36.9.
- This overhead is the SAE's own reconstruction error — irreducible regardless of quantisation.
- Thus, every other row in Table 5 is to be interpreted relative to the score of ≈60, not ≈37.
- The question each remaining row answers is: how much *additional* damage does quantisation add on top of the SAE's existing imperfection?

8-bit per-tensor is effectively lossless

- The differences in perplexity between 8-bit per-tensor and the identity patch are within the noise threshold.
- At 8-bit, the step size is fine enough that the quantisation error is negligible relative to

the SAE's own reconstruction error.

- This is to be expected; it serves as a sanity check.

Per-feature quantisation significantly outperforms per-tensor at low bitwidths

Table 6. Per-tensor vs. per-feature quantisation perplexity at $m = 512$. The same trend is observed for $m = 1024$.

Bits	Per-tensor PPL	Per-feature PPL	Difference
8	59.65	61.76	Per-feature slightly worse
4	92.33	63.63	Per-feature 31% better
2	2289	109.07	Per-feature 21× better

A likely explanation is the outlier activations observed in the data. A high pre-norm maximum (≈1917) forces per-tensor to use a very coarse quantisation step, mapping most activations to a handful of quantisation levels. Because per-feature calculates steps per dimension, dimensions with small ranges get proportionally fine granularity.

A counterintuitive observation: at $m = 512$, 8-bit per-tensor performs **better** than 8-bit per-feature, although this disappears at $m = 1024$.

Vector quantisation performs terribly

Both VQ-256 and VQ-1024 consistently demonstrate worse performance than even 2-bit regular quantisation across both m values.

Reasoning:

- Vector quantisation works fundamentally differently from uniform quantisation. Instead of quantising each dimension independently, it treats the entire activation vector as a single unit and replaces it with the nearest entry in a learned set.
- With 256 codebook entries, every token's 512-dimensional activation vector gets mapped to one of exactly 256 possible vectors.
- The problem here is a simple combinatorial mismatch. The number of possible activation patterns is very large (at least 2^{512}). This cannot be captured by such a small set of learned vectors.

Despite this, VQ has high CKA values.

- This reveals an important property of CKA: it measures whether the overall *relational structure* between tokens is preserved.
- Because VQ uses k-means, tokens which were similar to each other in the full-precision space tend to be assigned the same or nearby codebook entries.
- This undermines the importance of CKA as a metric. It should be treated as a supplementary geometric diagnostic rather than a proxy for model performance.

Identity patch improvement in $m = 1024$

For $m = 1024$, the identity patch overhead dropped from +62.2% to +27.6% relative to $m = 512$. This means this SAE has converged significantly better. Consequently, every other metric is also better for $m = 1024$.

Reasoning:

- The wider dictionary is more robust to quantisation.
- It distributes information across more features, each with smaller activation magnitudes, producing finer quantisation steps at every bitwidth.

2.3. Representation Damage and Causal Importance

2.3.1. Ranking Most Affected Neurons

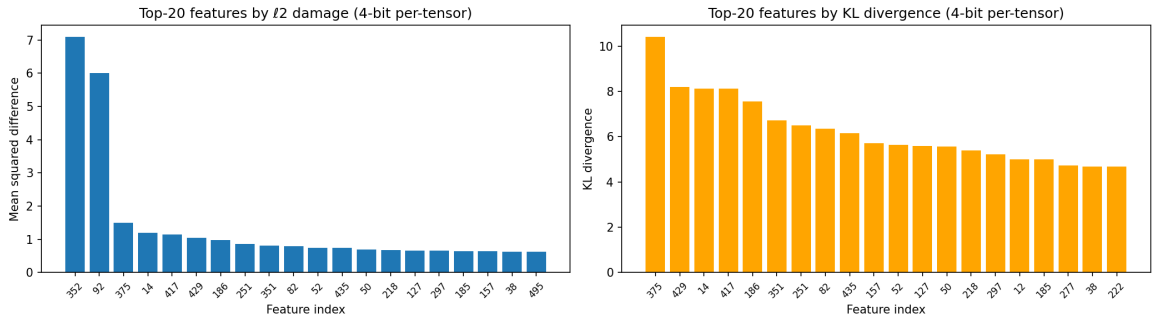

Figure 20. Top-20 most damaged features ranked by ℓ_2 damage (left) and by KL divergence (right), under 4-bit per-tensor quantisation on the $m = 512$ SAE. The two metrics agree on features 352 and 92 overall, but disagree near the top of the KL ranking.

Observations and reasoning

- The two metrics agree on the broad picture, but disagree near the 'top'.
  - The ℓ_2 ranking is dominated by features 352 and 92, which stand far above everything else — they are 4–5× more damaged than anything else.
  - The KL ranking tells a different story at the top. Feature 375 has the highest KL divergence, while features 352 and 92 do not appear in the top positions.
- Practical implications:
  - The two metrics are capturing different aspects of damage.
  - ℓ_2 identifies features where the numerical values are most displaced — relevant for understanding which features introduce the most reconstruction error.
  - KL identifies features where the distributional character is most changed — relevant for understanding which features have their functional behaviour most altered, since a feature that was previously active on specific tokens but is now always zero has changed its role in the network entirely.

2.3.2. Spectral Analysis

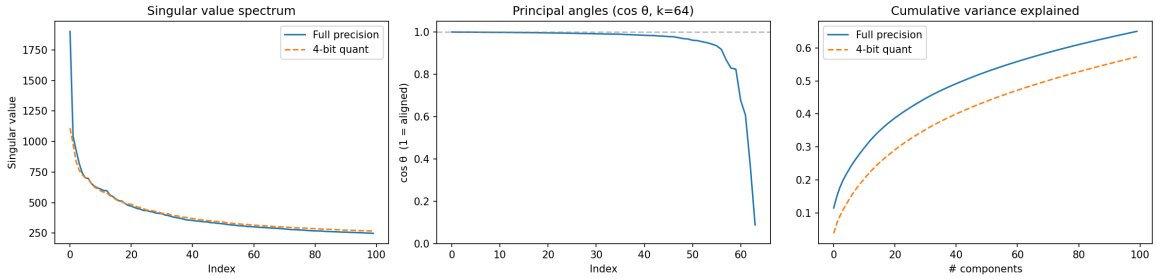

Figure 21. Spectral analysis of $m = 512$ SAE bottleneck activations under 4-bit per-tensor quantisation. *Left:* Singular value spectrum (full-precision vs 4-bit). *Centre:* Principal angles ($\cos\theta$, $k = 64$). *Right:* Cumulative variance explained by the top-k principal directions.

Observations and reasoning

- **Singular value spectrum:**
  - The full-precision and 4-bit spectra are nearly identical across all 100 shown components.
  - This tells us that 4-bit per-tensor quantisation does not dramatically reshape the variance structure of the bottleneck activations. The relative importance of each principal component is approximately preserved.
  - The damage is not concentrated in a few singular directions being amplified or destroyed.
- **Principal angles:**
  - The first ≈55 principal angles have cosines ≈1. After this, there is a sharp drop to near orthogonality around angle 64.
  - This indicates that the high-variance principal directions are very robust to 4-bit quantisation, while the alignment breaks down for the low-variance components.
  - For the small-variance components, the 4-bit quantisation noise, while small in absolute magnitude, is large enough relative to their variance to **completely** reorient them. These components were likely capturing fine-grained context which could not be resolved post-quantisation.
- **Cumulative variance:**
  - The full-precision curve consistently lies above the 4-bit curve across all components.
  - The gap indicates that quantisation redistributes variance. The quantised activations are less spread out — tokens that were distinct at full precision have been pushed closer together.
  - The two curves have very similar shapes, barring the shift. This is what one would expect from a uniform quantisation scheme applied independently to each dimension.

2.3.3. Top Activating Tokens and Ablation Study

Table 7. Top activating tokens and interpretations for the five most ℓ_2-damaged features at 4-bit per-tensor quantisation ($m = 512$).

Feature	ℓ_2 rank	Top activating tokens	Max act.	Interpretation
352	1	`<|endoftext|>,' kind','DB','49'`	83.44	Special/boundary token detector
92	2	`<|endoftext|>,' kind','don',' Twitter'`	82.12	Special/boundary token detector
375	3	`'</','29'),',' 10','37'`	8.19	Syntactic closing / numeric tokens
14	4	`' procedures', ' services', ' synchronization'`	9.16	Abstract noun / formal register
417	5	`' their',' existing',' to', ' with'`	8.84	High-frequency function words

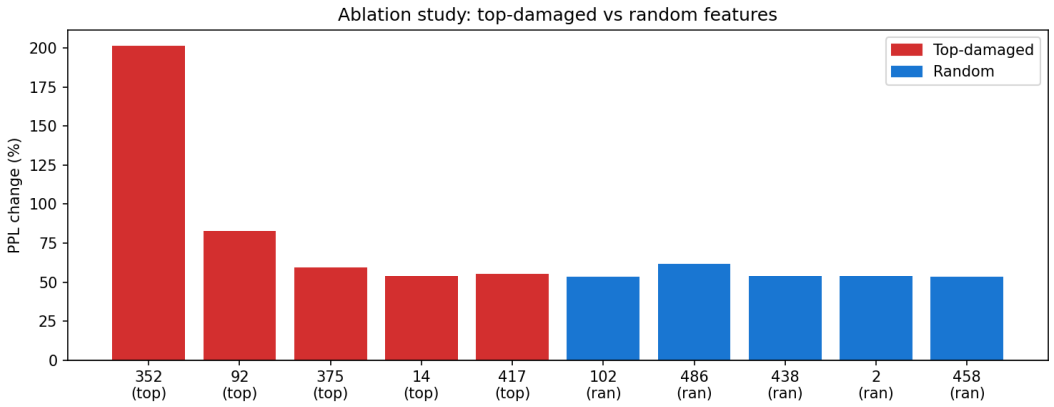

Figure 22. Ablation study: ΔPPL% when each feature is individually zeroed out. Red bars: top-5 most ℓ_2-damaged features. Blue bars: five randomly selected features. Feature 352 is a clear outlier (+201%). The remaining top-damaged features cluster near the random baseline, suggesting that high ℓ_2 damage does not reliably predict high functional importance.

Observations and reasoning

- Features 352 and 92 are 'sink features' — features that absorb a disproportionate share of the model's representational capacity for special or boundary tokens.
  - They are likely most damaged because, while they are high-magnitude, their semantic role (e.g., detecting a single special token) is narrow.
  - Ablating them causes a large PPL increase because the model has no redundant mechanism for handling such boundaries.
- The remaining three features notably have activations that are an order of magnitude lower than the top 2. They seem to encode genuine linguistic content.
  - These show smaller ablation impacts, similar to the random baseline.

2.3.4. Alignment Test

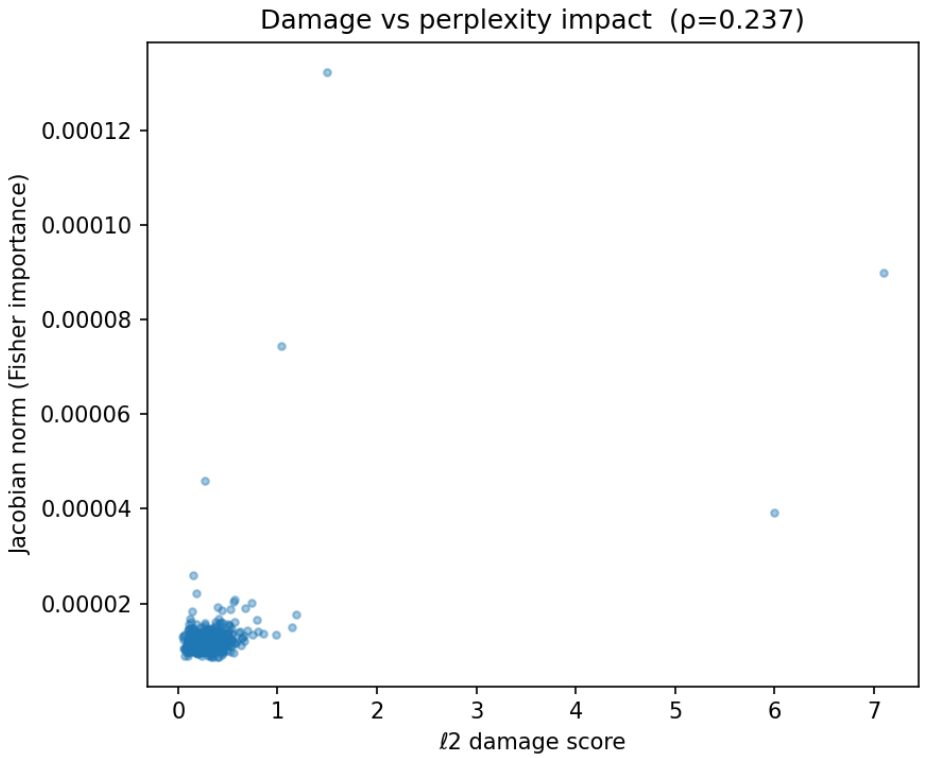

Figure 23. Scatter plot of ℓ_2 damage score vs. Jacobian norm (Fisher importance) for all 512 features of the $m = 512$ SAE. Spearman $\rho = 0.237$. The weak positive correlation indicates that quantisation damage and functional importance are largely unrelated properties across the dictionary.

Observations and reasoning

- The vast majority of the 512 features are clustered in the bottom-left corner — the low ℓ_2 damage and low Jacobian norm region. This implies that the bulk of the dictionary is behaving as expected: most features are neither particularly damaged by quantisation, nor particularly important for the model's predictions.
- There is a small number of outliers scattered from the cluster in distinct directions.
  - The few features towards the right of the graph are those which are both relatively important **and** relatively heavily damaged.
  - The few features towards the top of the graph are those which are important but **not** heavily damaged. This is strong evidence of weak alignment — the features the model relies on are **not** the ones most damaged by quantisation.
- Overall, the weak positive correlation is likely a consequence of the outliers which pull the regression line; in their absence, the correlation would have been even weaker.
- **Bottom line:** Quantisation damage and functional importance are largely unrelated properties across the dictionary.

2.4. Mechanistic Explanation and Robust Quantisation

The following findings have already been presented in prior sections and are not repeated here:

- Destructive interference between sparsity and quantisation — addressed in the alignment test (Section 2.3.4).
- Subspace rotation — addressed in the spectral analysis (Section 2.3.2).

2.4.1. Low Variance Collapse

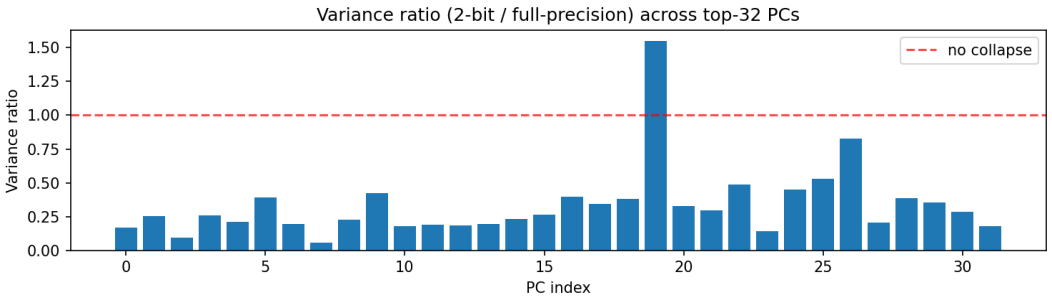

Figure 24. Variance ratio (2-bit quantised / full-precision) across the top 32 principal components of the $m = 512$ SAE bottleneck activations. The dashed red line at 1.0 marks the no-collapse reference. Nearly all components fall well below this line, with one anomalous component at PC index 19 exceeding it.

Observations and reasoning

- **General trend: non-uniform collapse.**
  - The collapse is not uniform across the 32 principal components.
  - There is no obvious trend based on PC index — the collapse is **not** simply more severe for low-variance directions.
  - This suggests that the collapse is driven by the interaction between the quantisation grid and the specific activation patterns in each direction, rather than by some simple relationship with the magnitude of the singular values.
  - Practical implication: at 2-bit quantisation, the model's layer 3 representations suffer a sharp loss in variance in essentially every direction. The downstream layers receive a much weaker signal, which is verified by the large increases in perplexity.
- **Anomaly at PC index 19:**
  - The bar crossing the Y-axis value of 1 indicates that 2-bit quantised activations actually have *more* variance in this direction than their full-precision counterparts.
  - The underlying mechanism is likely quantisation noise: when activations are rounded to coarse levels, the rounding errors are not uniformly distributed across all directions. They depend on how the quantisation grid aligns with the principal structure of the data. If many tokens have activations that fall near quantisation boundaries in

a particular direction, rounding can push them to opposite sides of the boundary, artificially inflating the spread of values in that direction.

- This happens in only 1 of 32 directions, allowing us to treat it as an anomaly.

2.4.2. Sparsity Fragility

Table 8. Sparsity fragility metrics for the $m = 512$ SAE ($k = 51$) under per-tensor quantisation. L0 is the mean number of active features per token; Jaccard similarity measures the overlap of active feature sets before and after quantisation.

Metric	Full precision	4-bit	2-bit
L0 (active features/token)	51.0	13.6	0.3
Jaccard similarity	1.000	0.267	0.006
False negatives/token	0	37.37	50.69
False positives/token	0	0.00	0.00

Observations and reasoning

- **The dominant failure mode is erasure, not corruption.**
  - Zero false positives across both bitwidths means quantisation never erroneously activates a feature that should be silent; it only kills features that should be active.
  - This has a clear explanation:
    - Owing to a coarse step size, small positive values are rounded to zero.
    - Because ReLU is applied before top-k selection, there is no symmetric upward rounding of zeroes that might have led to a false positive.
- **The severity of degradation is extreme.**
  - At 4-bit, 37 of 51 features are erroneously killed, while at 2-bit, essentially all of them are.
  - The very low Jaccard similarity at 2-bit implies that the active feature sets before and after quantisation share virtually nothing in common.
- **This explains perplexity better than other findings.**
  - The extremely large jump in PPL values for 2-bit per-tensor is not caused by numerical *imprecision*. It is caused by the active features themselves being erased entirely.
  - The model downstream is not receiving a noisy version of the intended representation — it is receiving a nearly empty one.
  - This is why perplexity is so much worse than one would predict based on SDS or MSE values alone.
- **This is likely specific to top-k SAEs, and not a general quantisation phenomenon.**
  - This failure mode is directly caused by the interaction between top-k sparsity and uniform quantisation.
  - A dense representation, where all directions carry a meaningful signal, would not

suffer this failure because no values would be concentrated near zero.

– The sparsity that makes the SAE interpretable is precisely what makes it fragile to quantisation.

2.4.3. Subspace Preserving Quantisation

Table 9. Subspace-preserving quantisation vs. standard 4-bit per-tensor quantisation for the $m = 512$ SAE. Subspace-preserving quantises the projection onto the top-32 singular vectors at 8-bit and the residual at 2-bit.

Method	PPL	ΔPPL%	SDS	CKA	MSE
Standard 4-bit	90.65	+141.4%	0.1009	0.8475	0.3182
Subspace-preserving (8+2)	103.25	+174.9%	0.1077	0.8312	0.3638

The ΔPPL% values in Table 9 are relative to the **clean baseline** (≈36.9), not the identity patch.

Observations and reasoning

Subspace-preserving quantisation performs worse on every metric.

Likely reasoning:

- The subspace-preserving approach decomposes each activation vector into two components: the projection onto the top-k singular vectors (the important subspace) and the residual. The residual is then quantised at 2-bit.
- This decomposition assumes that information is concentrated in the important subspace and is negligible in the residual.
- This assumption is reasonable for a dense representation, but fails for a top-k sparse one — in the latter case, sparse active features are distributed across both components. The top-k constraint creates a representation where importance is determined by *which features are active*, not by which directions carry the most variance.

References

A. Karpathy. nanoGPT, 2022. URL https://github.com/karpathy/nanoGPT. GitHub repository.

A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

R. Xiong, Y. Yang, D. He, K. Zheng, S. Zheng, C. Xing, H. Zhang, Y. Lan, L. Wang, and T. Liu. On layer normalization in the transformer architecture. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, pages 10524–10533. PMLR, 2020.